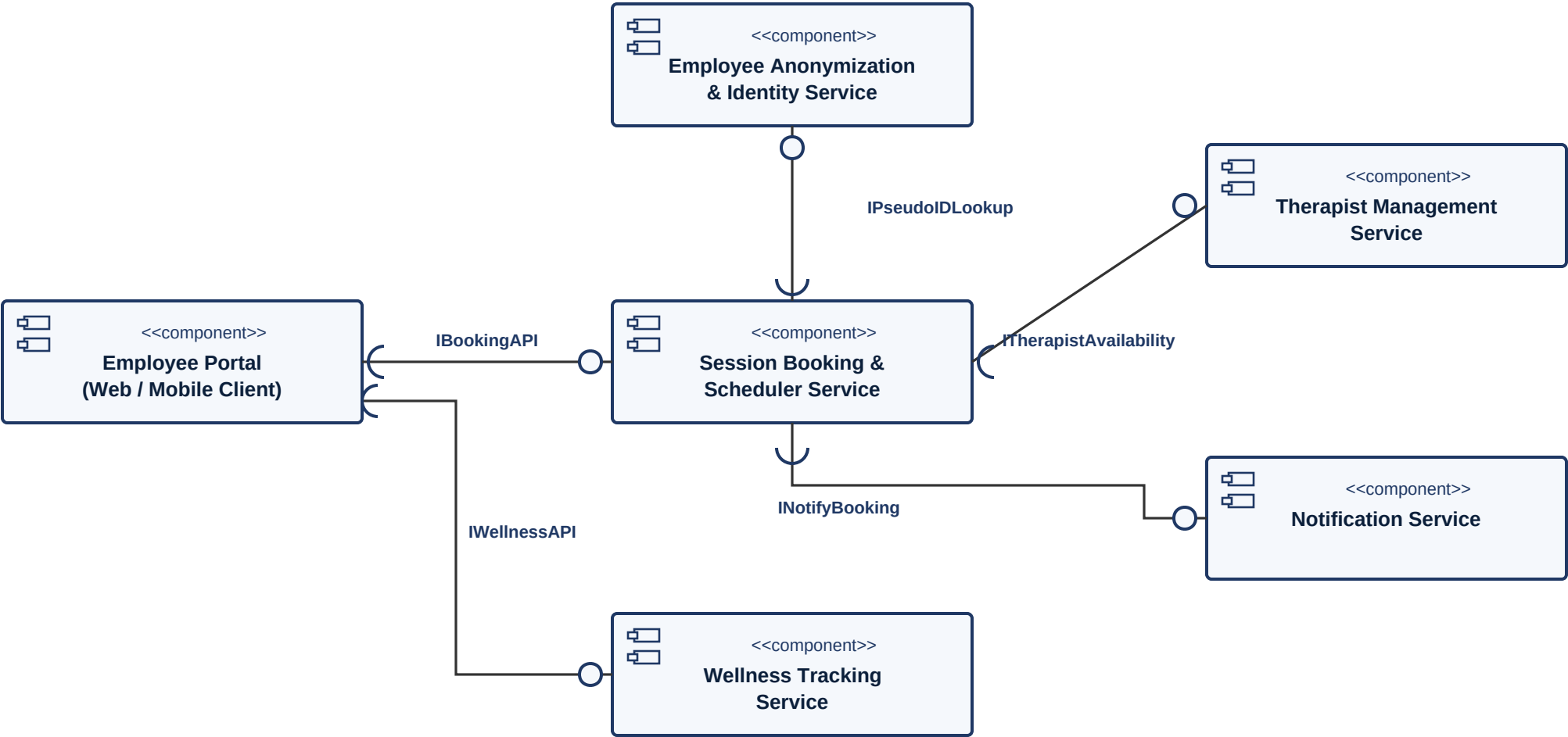


Component Diagram — Corporate Mental Wellness & Counseling Platform

Architecture Style: Microservices Architecture



Legend

- Provided Interface (ball)
- ⌋ Required Interface (socket)
- Communication link
- Component icon

Security Note

The Therapist Management Service and Notification Service never connect directly to the Anonymization Service. Only the Session Booking Service consumes `IPseudoIDLookup`, so real employee identity is never exposed outside that boundary.

Lab 3 – Architectural Pattern Selection: Written Justification

Scenario 14 – Corporate Mental Wellness & Counseling Platform

Name: Avaneesh N **SRN:** PES1UG24AM917

Scenario

The system is an anonymous counseling portal for a company, where employees can check their stress levels, book sessions with a therapist, and keep track of their wellness data. The main requirement is that the therapist should never find out who the employee actually is – the system only shows a pseudo-ID. It also needs to handle bookings across different time zones since employees could be in different office locations, and it has to stay secure and fast even when a lot of people are using it at once.

Architecture Selection

We chose Microservices Architecture for the Corporate Mental Wellness & Counseling Platform.

Reason 1

Since the whole point of the platform is that employee identity stays hidden from the therapist, it made sense to keep the identity/anonymization logic in its own separate service instead of mixing it into everything else. In a microservices setup, only the Session Booking service ever talks to the Anonymization service to get a pseudo-ID – the Therapist and Notification services never see it at all. If this was built as one big monolithic app instead, the identity data would end up being reachable from a lot more places in the code, which is exactly what we want to avoid here.

Reason 2

Not all parts of this system get used the same way. Booking a session needs to be fast and reliable at all times (especially with people across time zones booking at once), while something like the Wellness Tracker or Notifications doesn't get nearly as much traffic. With microservices, each of these can be scaled up or down on its own, and if one of them (say, Notifications) goes down for a bit, employees can still log in, check their stress score and book a session normally. That kind of isolation is hard to get with a single client-server setup where everything usually goes down together.

Security Advantage

Because only one service (Session Booking) is allowed to call the Anonymization service's IPseudoIDLookup interface, there's basically one narrow, easy-to-monitor point where real employee identity ever gets used. Even if the Therapist or Notification service had a bug or got compromised, there's no way for it to pull actual employee data, since it was never given access to that interface in the first place. This lines up with the FR-001 requirement that the therapist dashboard should only ever show a pseudo-ID.

Performance Benefit

The Session Booking service, which is the part that has to deal with timezone conversion and stays under the most load (NFR-001), can be scaled up independently without having to scale the entire system with it. So during busy hours we can just add more resources to booking and scheduling, while lighter services like Wellness Tracking keep running as they normally would. That keeps booking fast without wasting resources on parts of the system that don't need it.